

Alexandria University
Faculty of Engineering
Computer and Systems Engineering Department

Lab 3: Paradigms Blender
Smart Study Advisor System

Course: Programming Languages Paradigms – CSE-225

#	Name	Student ID
1	Mohamed Abdalla Yassin Mohamed	23010765
2	Mohamed Mahmoud Ahmed Mohamed	23010780
3	Mohamed Gomaa AbdElzaher	23010734
4	Moaz Hassan Ali	23010858

GitHub Repository:

<https://github.com/Mohamed-Abdalla-Yassen/Smart-Study-Advisor.git>

Contents

1	Introduction	2
2	System Architecture	2
2.1	Overall Pipeline (Non-AI Version)	2
2.2	Component Overview	2
3	Architecture: Where Each Paradigm Is Used	2
3.1	Logic Paradigm – Prolog	2
3.2	Object-Oriented Paradigm – Django Models & Views	3
3.3	Functional Paradigm – Data Transformation	3
3.4	Imperative Paradigm – Execution Flow	3
4	Part 1: AI-Based Advisor	3
5	Part 2: Non-AI Advisor	4
6	Reflection	4
7	Conclusion	4

Introduction

This report documents the design and implementation of a **Smart Study Advisor** system, developed as part of Lab 3 for the Programming Languages Paradigms course (CSE-225). The project showcases how different programming paradigms can be combined into a single cohesive intelligent system.

Two versions of the system were built:

- **AI-Based Advisor** – integrates an external AI API (e.g. Gemini) to generate course recommendations via structured prompts.
- **Non-AI Advisor** – uses Logic (Prolog), Functional, Object-Oriented, and Imperative paradigms to implement rule-based reasoning without an AI back-end.

System Architecture

Overall Pipeline (Non-AI Version)

Mobile App → Django Backend → Prolog Inference Engine → Django →
Mobile App

The student interacts through a mobile interface. The Django backend coordinates the data flow and applies functional transformations before forwarding queries to the Prolog engine. Prolog reasons over the knowledge base and returns recommendations that are sent back to the user.

Component Overview

Component	Description
Mobile App	Simple UI built with Flutter/React Native for submitting preferences and displaying recommendations.
Django Backend	REST API server that handles requests, applies data transformations, and bridges the mobile app with the Prolog engine.
Prolog Engine	Inference engine containing rules for course difficulty, prerequisites, and student preferences. Exposes a <code>recommend/2</code> predicate.

Architecture: Where Each Paradigm Is Used

Logic Paradigm – Prolog

Where used: Inference engine layer.

Why it fits: Prolog naturally encodes knowledge as facts and rules, making it ideal for expressing academic relationships like course prerequisites and difficulty levels. The query `recommend(Student, Course)` elegantly captures the entire recommendation logic in a declarative form that would require complex conditional trees in other paradigms.

```
difficulty(algorithms, hard).  
prerequisite(algorithms, data_structures).  
  
recommend(Student, Course) :-  
    passed(Student, Prereq),  
    prerequisite(Course, Prereq),  
    prefers(Student, difficulty(Course)).
```

Listing 1: Sample Prolog rules

Object-Oriented Paradigm – Django Models & Views

Where used: System design, data models, and API views.

Why it fits: OOP provides a clean way to model real-world entities such as `Student`, `Course`, and `Recommendation` as classes with encapsulated state and behaviour. Django’s class-based views also leverage inheritance and polymorphism to reduce boilerplate.

Functional Paradigm – Data Transformation

Where used: Pre-processing student input and post-processing Prolog results within the Django layer.

Why it fits: Functional operations such as `map`, `filter`, and `reduce` produce clean, side-effect-free transformations of course lists and student preference data. This makes the transformation pipeline easy to test and reason about.

Imperative Paradigm – Execution Flow

Where used: Request handling, authentication checks, database queries, and subprocess calls to the Prolog interpreter.

Why it fits: Sequential, step-by-step control flow is the most natural way to express I/O-heavy operations such as reading a request, querying a database, spawning a subprocess, and writing a response.

Part 1: AI-Based Advisor

The AI-based version integrates the **Gemini API** (or equivalent) to provide course recommendations. A structured prompt containing the student’s completed courses, GPA, and stated preferences is sent to the model. The model returns a ranked list of recommendations with brief justifications.

Key design decisions:

- Prompts are templatised in Django to ensure consistent structure.
- Responses are parsed and normalised before being returned to the mobile app.
- Error handling accounts for API rate limits and malformed responses.

Part 2: Non-AI Advisor

The non-AI version relies entirely on the multi-paradigm pipeline described above. The Prolog knowledge base encodes all domain knowledge, while Django handles communication and data transformations. No external AI service is involved.

Key design decisions:

- Prolog is invoked as a subprocess from Django using Python's `subprocess` module.
- Results are returned as JSON and displayed identically to the AI version, allowing fair comparison.
- The knowledge base is modular, making it easy to extend with new courses or rules.

Reflection

If you were building a real AI system, what approach would you choose and why?

For a real-world production system, we would adopt a **hybrid approach** rather than relying on a single paradigm or solely on a large language model (LLM).

A **Prolog-based reasoning core** would handle hard constraints such as prerequisites and academic policies, guaranteeing correctness and auditability. An **LLM integration** (similar to Part 1) would handle the soft, preference-based aspects of recommendations — areas where natural language understanding adds value beyond rigid rules.

The **functional paradigm** would remain central to data pipelines (transforming student records, normalising API responses), and **OOP** would continue to provide the structural backbone for the backend. Pure **imperative** code would be confined to I/O and orchestration.

This blend ensures transparency (rules can be inspected and audited), flexibility (the LLM handles edge cases gracefully), and maintainability (each paradigm is used only where it is strongest).

Conclusion

This lab demonstrated that no single paradigm is universally optimal. Logic programming excels at rule-based reasoning; OOP provides clean structure; functional programming ensures safe data transformation; and imperative code drives execution. Combining all four produced a system that is more robust, readable, and maintainable than any single-paradigm alternative.